# RUN: Microprocessor Interfacing, Interrupts and Timing

Nov 6, 2026 · @shahan

Everything the Microprocessor Sessional-2 has asked for five years running, solved where it belongs. Allow about 2 hours. **Every delay count was recomputed.**

## How this run works

**Scope assumption:** the group said "whatever sir taught". The Sessional-2 papers from 2021, 2022, 2024 and 2025 ask the same four things almost word for word, so this run covers those four, plus the delay-counter program from 2023 and the endsem.

| Topic | 2021 | 2022 | 2023 | 2024 | 2025 |
|---|---|---|---|---|---|
| I/O-mapped or memory-mapped interfacing | Q1 | Q1 | | Q1 | Q1 |
| Timing diagram (LDA or STA) | Q2 | Q2 | | Q2 | |
| Interrupts, INTR, RIM and SIM | | Q3 | | Q3 | Q2 |
| 8086 vs 8085 | Q3 | | | | Q3 |
| Delay and counter program | | | Q1 | | |

If a topic shows up in four of five years, it's not a prediction, it's a tradition.

**Videos:** Bharat Acharya Education on YouTube for the 8085 timing diagrams, interrupts and 8255. His full notes are already in `Microprocessor/Notes/`, so use the PDFs for the diagrams you'll copy into the exam.

## Stop 1: 8086 vs 8085

> **PYQ, Sessional-2 2025 Q3, 3 marks; Sessional-2 2021 Q3; Endsem 2022 Q1(b); Endsem 2024 Q1(b).** Explain how the 8086 microprocessor is different from the 8085.

**Answer:**

| Basis | 8085 | 8086 |
|---|---|---|
| Word size | 8-bit | 16-bit |
| Data bus | 8 lines | 16 lines |
| Address bus | 16 lines, so 64 KB memory | 20 lines, so 1 MB memory |
| Memory organisation | Linear | Segmented: CS, DS, SS, ES |
| Pipelining | None | 6-byte instruction queue, so it fetches while executing |
| Internal units | One unit | BIU (bus interface) and EU (execution), working in parallel |
| Flags | 5 (S, Z, AC, P, CY) | 9: 6 status plus 3 control (TF, IF, DF) |
| Clock | 3 MHz | 5 to 10 MHz |
| Multiply and divide | No instructions | MUL, DIV, IMUL, IDIV |
| I/O ports | 256 (8-bit address) | 65,536 (16-bit address) |
| Modes | One | Minimum and maximum mode |
| Pins | 40, address/data multiplexed on AD0 to AD7 | 40, multiplexed on AD0 to AD15 |

The 1 MB figure comes from the 20-bit physical address:

$$\text{Physical address} = \text{Segment} \times 10_H + \text{Offset}, \qquad 2^{20} = 1\ \text{MB}$$

**The 8086 is a 16-bit, pipelined, segmented processor with a 1 MB address space. The 8085 is 8-bit, non-pipelined and limited to 64 KB.**

## Stop 2: Machine cycles and timing diagrams

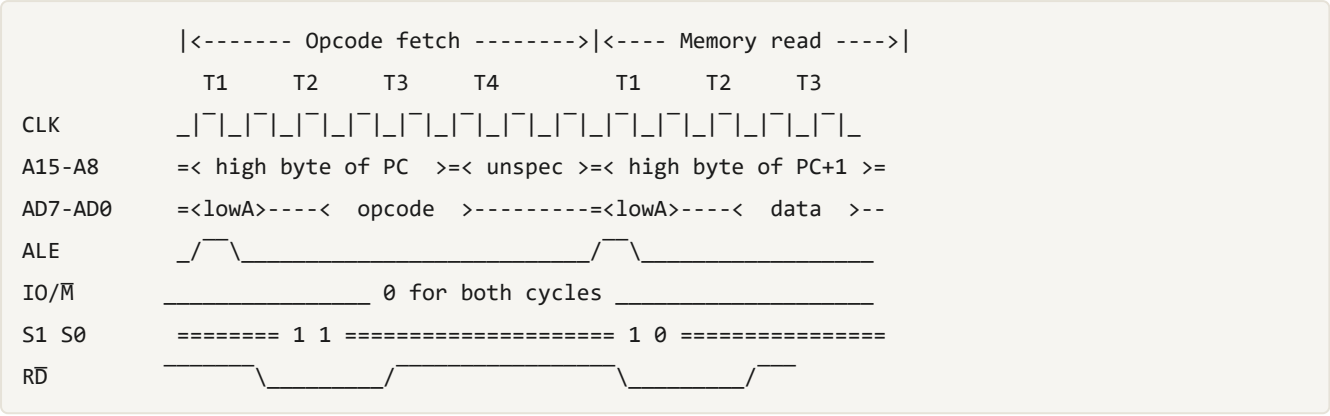
An instruction is made of **machine cycles**, and each machine cycle is made of **T-states** (clock periods). A timing diagram just draws the bus signals T-state by T-state.

| Machine cycle | T-states | IO/$\bar{M}$ | S1 S0 | Strobe |
|---|---|---|---|---|
| Opcode fetch | 4 (6 for some) | 0 | 1 1 | R$\bar{D}$ low |
| Memory read | 3 | 0 | 1 0 | R$\bar{D}$ low |
| Memory write | 3 | 0 | 0 1 | W$\bar{R}$ low |
| I/O read | 3 | 1 | 1 0 | R$\bar{D}$ low |
| I/O write | 3 | 1 | 0 1 | W$\bar{R}$ low |

**What happens in each T-state of a cycle:**

- **T1:** the address goes out. The high byte sits on A15 to A8, and the low byte on AD7 to AD0. ALE pulses high so an external latch (74LS373) grabs the low byte before the lines are reused for data.
- **T2:** R$\bar{D}$ (or W$\bar{R}$) goes low, and AD7 to AD0 now carry data.
- **T3:** the data is read in (or written), and R$\bar{D}$ goes back high.
- **T4**, opcode fetch only: the opcode is decoded, and the buses are idle.

**The one waveform to memorise**, an opcode fetch followed by a memory read:




Every other diagram is this, with a different row of cycles.

> **PYQ, Endsem 2025 Q1(c), 4.5 marks; Sessional-2 2024 Q2, 5 marks; Sessional-2 2022 Q2.** Draw and explain the timing diagram of LDA 2150H, stored at address 2000H.

**Answer:** LDA is a 3-byte instruction: 2000H holds 3AH (the opcode), 2001H holds 50H, and 2002H holds 21H, because the address is stored low byte first. It needs 4 machine cycles.

| Cycle | T-states | Address out | Data on AD7-AD0 | IO/$\overline{M}$ | S1 S0 | Purpose |
|---|---|---|---|---|---|---|
| M1: Opcode fetch | 4 | 2000H | 3AH | 0 | 1 1 | Fetch and decode LDA |
| M2: Memory read | 3 | 2001H | 50H | 0 | 1 0 | Low byte of the address |
| M3: Memory read | 3 | 2002H | 21H | 0 | 1 0 | High byte of the address |
| M4: Memory read | 3 | 2150H | contents of 2150H | 0 | 1 0 | Data into the accumulator |

Draw the waveform above with four cycles. In each cycle ALE pulses in T1, the address splits as high byte on A15-A8 (20H, 20H, 20H, then 21H) and low byte on AD7-AD0 (00H, 01H, 02H, then 50H), and RD$\overline{}$ goes low in T2 and T3.

$$\text{Total} = 4 + 3 + 3 + 3 = 13 \text{ T-states}$$

**LDA 2150H = opcode fetch + 3 memory reads = 13 T-states. The last read brings the contents of 2150H into A.** For LDA 2500H, just change the M4 address to 2500H.

> **PYQ, Sessional-2 2021 Q2.** Draw and explain the timing diagram of STA 2500H.

**Answer:** STA is the mirror image of LDA. The first three cycles are identical (opcode 32H, then 00H, then 25H), but the last one **writes** the accumulator to 2500H.

| Cycle | T-states | Address out | Data | S1 S0 | Strobe |
|---|---|---|---|---|---|
| M1: Opcode fetch | 4 | PC | 32H | 1 1 | RD$\overline{}$ |
| M2: Memory read | 3 | PC + 1 | 00H | 1 0 | RD$\overline{}$ |
| M3: Memory read | 3 | PC + 2 | 25H | 1 0 | RD$\overline{}$ |
| M4: Memory write | 3 | 2500H | contents of A | 0 1 | WR$\overline{}$ low |

**STA 2500H = opcode fetch + 2 memory reads + 1 memory write = 13 T-states.** The key difference to draw: in M4, $\overline{WR}$ goes low instead of $\overline{RD}$.

> **PYQ, Endsem 2024 Q1(c).** Draw and explain the timing diagram of IN 68H.

**Answer:** IN is a 2-byte instruction (DBH, then 68H), with 3 machine cycles.

| Cycle | T-states | Address out | Data | IO/$\overline{M}$ | S1 S0 |
|---|---|---|---|---|---|
| M1: Opcode fetch | 4 | PC | DBH | 0 | 1 1 |
| M2: Memory read | 3 | PC + 1 | 68H | 0 | 1 0 |
| M3: I/O read | 3 | 6868H | data from port 68H | **1** | 1 0 |

In the I/O read cycle the 8-bit port address appears on **both** halves of the address bus, so A15-A8 = 68H and AD7-AD0 = 68H, and IO/$\overline{M}$ goes high.

**IN 68H takes 10 T-states. The giveaway in the diagram is IO/$\overline{M}$ = 1 in M3, with the port address duplicated.**

## Stop 3: The 8085 interrupts, RIM and SIM

An interrupt makes the processor pause its program, run a service routine (ISR), then come back. It's how a keyboard gets attention without the CPU checking it a million times a second.

| Interrupt | Priority | Maskable? | Triggering | Vector address |
|---|---|---|---|---|
| TRAP (RST 4.5) | 1, highest | No | Edge and level | 0024H |
| RST 7.5 | 2 | Yes | Rising edge (latched) | 003CH |
| RST 6.5 | 3 | Yes | Level | 0034H |
| RST 5.5 | 4 | Yes | Level | 002CH |
| INTR | 5, lowest | Yes | Level | Not vectored: supplied by external hardware |

The vector address of RST n is n × 8, in hex:

$$\text{RST } 7.5 \rightarrow 7.5 \times 8 = 60 = 003C_H, \qquad \text{RST } 5.5 \rightarrow 5.5 \times 8 = 44 = 002C_H$$

- **EI** enables all the maskable interrupts, and **DI** disables them. TRAP ignores both.
- **Software interrupts** RST 0 to RST 7 are instructions, jumping to 0000H, 0008H, and so on up to 0038H.

**SIM (Set Interrupt Mask)**: load a control byte into A, then execute SIM.

| Bit | D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|---|---|---|---|---|---|---|---|---|
| SIM | SOD | SDE | x | R7.5 | MSE | M7.5 | M6.5 | M5.5 |

M = 1 masks (disables) that interrupt. MSE = 1 makes the mask bits take effect. R7.5 = 1 resets the RST 7.5 flip-flop. SDE = 1 sends bit D7 out on the SOD pin (serial out).

**RIM (Read Interrupt Mask)**: after RIM, A holds the status.

| Bit | D7 | D6 | D5 | D4 | D3 | D2 | D1 | D0 |
|---|---|---|---|---|---|---|---|---|
| RIM | SID | I7.5 | I6.5 | I5.5 | IE | M7.5 | M6.5 | M5.5 |

I = 1 means that interrupt is pending. IE shows whether interrupts are enabled. SID is the bit read from the serial input pin.

> **PYQ, Sessional-2 2024 Q3, 5 marks; Endsem 2024 Q4(a).** Explain the use of RIM and SIM instructions, with an example.

**Answer:** SIM sets the masks for RST 7.5, 6.5 and 5.5, resets the RST 7.5 latch, and sends serial output. RIM reads the masks, the pending interrupts, the interrupt-enable flag and serial input. Use the two tables above.

Example 1: enable RST 6.5 only, masking 7.5 and 5.5.

```
MVI A, 0DH    ; 0000 1101: MSE = 1, M7.5 = 1, M6.5 = 0, M5.5 = 1
SIM           ; apply the mask
EI            ; enable interrupts
```

Example 2: if RST 7.5 is pending, call its routine.

```
RIM           ; A <- interrupt status
ANI 40H       ; keep D6, the I7.5 bit
JNZ SERV75    ; pending, so service it
```

Example 3: send a 1 out on the SOD line.

```
MVI A, C0H    ; 1100 0000: SOD = 1, SDE = 1
SIM
```

**SIM writes the masks and serial output; RIM reads the masks, the pending interrupts and serial input. Writing 0DH then executing SIM and EI leaves only RST 6.5 enabled.**

## Stop 4: INTR, single and multiple interrupts

INTR is the only **non-vectored** interrupt: the 8085 doesn't know where to jump, so the device has to tell it, by placing an instruction on the data bus.

**The INTR sequence:**

1. The device pulls INTR high.
2. The 8085 finishes the current instruction, checks INTR (if interrupts are enabled with EI), and disables further interrupts.
3. It replies with INTĀ (low), an acknowledge.
4. External hardware puts a one-byte **RST n** opcode on the data bus during INTĀ.
5. The 8085 executes it: it pushes the PC onto the stack and jumps to n × 8.
6. The ISR runs, ends with EI and RET, and the main program resumes.

The RST opcodes all follow one pattern, with n in the middle three bits:

$$\text{RST } n = 11\,NNN\,111_2, \qquad \text{RST } 5 = 11\,101\,111_2 = EF_H \rightarrow 0028_H$$

> **PYQ, Sessional-2 2025 Q2, 3 marks; Sessional-2 2022 Q3; Endsem 2025 Q4(a).** How is the INTR line of the 8085 used to handle (i) a single interrupt and (ii) multiple interrupts? Explain with a diagram.

**Answer:**

**(i) Single interrupt.** Hard-wire the opcode of one RST, say RST 5 (EFH), onto the data bus through a tri-state buffer (74LS244) whose enable is INTA̅. The device output drives INTR. When the 8085 acknowledges, the buffer puts EFH on the bus and the 8085 jumps to 0028H.

```
Device --------------------------------> INTR
                                                  8085
+5V/GND wired as 1110 1111 --> [74LS244] --> D7-D0
                                    ^
INTA (active low) -----------------+   enables the buffer
```

**(ii) Multiple interrupts.** Eight devices share the one INTR line. A **priority encoder** (74LS148) works out which device has the highest priority, and its 3-bit output becomes the NNN bits of the RST opcode.

```
I0 ... I7 from 8 devices (active low)
   |                          |
[74LS148 priority encoder]   +--[NAND / OR]--> INTR
   | A2 A1 A0 (NNN)
   v
[74LS244 buffer]: D7 D6 = 11, D5 D4 D3 = NNN, D2 D1 D0 = 111  --> data bus
   ^
INTA ---- enables the buffer
```

Any active device also drives INTR through the gate. During INTA̅ the buffer places 11 NNN 111 on the data bus, so the highest-priority device's RST is executed. If two devices interrupt together, the encoder picks the higher input. In practice the **8259A programmable interrupt controller** does all of this in one chip, with programmable priorities, and cascades to 64 levels.

**Single: a fixed RST opcode on the bus during INTA̅. Multiple: a priority encoder builds 11 NNN 111 so each device gets its own vector, or use the 8259A.**

## Stop 5: I/O-mapped and memory-mapped interfacing

The 8085 can treat a device either as an **I/O port**, reached with IN and OUT, or as a **memory location**, reached with any memory instruction. Interfacing means decoding the address so that exactly one device answers, and generating the right control signal.

**The four control signals**, made from IO/M̅, RD̅ and WR̅ with a few gates:

| Signal | Active when | Used by |
|---|---|---|
| MEM$\bar{R}$ | IO/$\bar{M}$ = 0 and R$\bar{D}$ = 0 | Memory read, memory-mapped input |
| MEM$\bar{W}$ | IO/$\bar{M}$ = 0 and W$\bar{R}$ = 0 | Memory write, memory-mapped output |
| IO$\bar{R}$ | IO/$\bar{M}$ = 1 and R$\bar{D}$ = 0 | IN instruction |
| IO$\bar{W}$ | IO/$\bar{M}$ = 1 and W$\bar{R}$ = 0 | OUT instruction |

> **PYQ, Sessional-2 2022 Q1, 5 marks.** Distinguish between I/O-mapped I/O and memory-mapped I/O.

**Answer:**

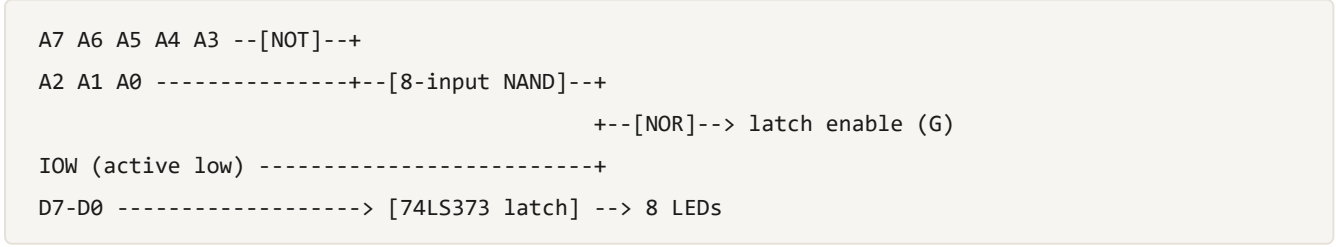
| Basis | I/O-mapped I/O | Memory-mapped I/O |
|---|---|---|
| Address size | 8-bit port address | 16-bit address |
| Number of devices | 256 inputs + 256 outputs | Shares the 64K space with memory |
| Instructions | Only IN and OUT | Any memory instruction: LDA, STA, MOV M, ADD M |
| Control signals | IO$\bar{R}$, IO$\bar{W}$ (IO/$\bar{M}$ = 1) | MEM$\bar{R}$, MEM$\bar{W}$ (IO/$\bar{M}$ = 0) |
| Data transfer | Only through the accumulator | Any register |
| Memory space | Not used up | Reduced by each device |
| Decoding hardware | Simpler, 8 address lines | More complex, 16 lines |
| Speed | IN/OUT take 10 T-states | LDA/STA take 13, but MOV M takes 7 |

**I/O-mapped uses an 8-bit port address, IN/OUT and IO$\bar{R}$/IO$\bar{W}$. Memory-mapped uses a 16-bit address, any memory instruction, and MEM$\bar{R}$/MEM$\bar{W}$.**

> **PYQ, Sessional-2 2024 Q1, 5 marks; Sessional-2 2021 Q1; Endsem 2025 Q4(b).** Explain how input and output devices are interfaced with the 8085 as I/O-mapped I/O, with a suitable diagram and example.

**Answer:** Take an **output port at 07H** driving 8 LEDs, and an **input port at 84H** reading 8 DIP switches.

**Output port 07H.** The port address appears on A7 to A0 during OUT 07H. A NAND-based decoder outputs 0 only for 0000 0111. That, combined with IO$\bar{W}$ through a NOR gate, produces the enable pulse for a 74LS373 latch, which holds the data from D7 to D0 and drives the LEDs.




```
MVI A, 0FH    ; pattern: light the lower four LEDs
OUT 07H       ; the address decodes, IOW pulses, the latch holds 0FH
```

**Input port 84H.** The decoder matches 1000 0100. Combined with IO$\bar{R}$, it enables a 74LS244 tri-state buffer, which puts the switch states on the data bus only during IN 84H. At all other times it stays off the bus.

```
IN 84H        ; A <- switch states
```

**Decode the 8-bit port address, gate it with IO$\bar{W}$ to clock a latch (output), or with IO$\bar{R}$ to enable a tri-state buffer (input).**

> **PYQ, Sessional-2 2025 Q1, 5 marks; Endsem 2022 Q4(a).** Explain how input and output devices are interfaced with the 8085 as memory-mapped I/O, with a suitable diagram and example.

**Answer:** The device gets a full **16-bit memory address**, say output at 8000H and input at 8001H. Decode all of A15 to A0 (or partially decode the high lines), and gate the result with **MEM$\bar{W}$** for output or **MEM$\bar{R}$** for input. The same latch and buffer circuits as above are used.

```
A15-A0 --[address decoder: match 8000H]--+
                                         +--[NOR]--> 74LS373 enable --> LEDs
MEMW (active low) ----------------------+

A15-A0 --[address decoder: match 8001H]--+
                                         +--[NOR]--> 74LS244 enable --> switches onto D7-D0
MEMR (active low) ----------------------+
```

```
MVI A, 0FH
STA 8000H     ; output: write to the LED latch
LDA 8001H     ; input: read the switches into A

LXI H, 8001H
MOV B, M      ; any register works, an advantage of memory mapping
```

**The device is just a memory address: decode 16 bits, gate with MEM$\bar{R}$/MEM$\bar{W}$, and use LDA/STA or MOV M.** The price is that those addresses can no longer hold memory.

## Stop 6: 8255 PPI

The 8255 **Programmable Peripheral Interface** gives you 24 I/O lines in one chip, configured by software, so you don't wire up latches and buffers by hand.

- **Ports:** A (8 lines), B (8 lines) and C (8 lines, split into upper and lower halves of 4).
- **Groups:** Group A = port A + upper C. Group B = port B + lower C.
- **Modes:** Mode 0 is simple I/O. Mode 1 is handshake (strobed) I/O, with port C giving the handshake lines. Mode 2 is bidirectional, port A only. **BSR mode** sets or resets individual bits of port C.
- **Selecting a register:** A1 and A0 choose the port. 00 is port A, 01 is port B, 10 is port C, and 11 is the control register. C$\bar{S}$ comes from the address decoder.

**Control word for I/O mode:**

| Bit | D7 | D6 D5 | D4 | D3 | D2 | D1 | D0 |
|---|---|---|---|---|---|---|---|
| Meaning | 1 = I/O mode | Group A mode | Port A | Port C upper | Group B mode | Port B | Port C lower |

For each port bit, 1 = input and 0 = output.

> **PYQ, Endsem 2022 Q4(b); Endsem 2024 Q4(c).** What is a PPI? How are peripherals connected to the microprocessor through the 8255? Explain with an example.

**Answer:** The 8255 sits between the 8085 and the devices. Its D7 to D0 connect to the data bus, R$\bar{D}$ and W$\bar{R}$ to IO$\bar{R}$ and IO$\bar{W}$, A1 and A0 to the address lines A1 and A0, and C$\bar{S}$ to the decoder output. The devices connect to ports A, B and C.

Example: port A as input from switches, port B as output to LEDs, everything in mode 0, with the 8255 at base address 80H (so A = 80H, B = 81H, C = 82H, control = 83H).

$$\text{CW} = 1\ 00\ 1\ 0\ 0\ 0\ 0_2 = 90_H$$

```
MVI A, 90H    ; control word: port A in, port B out, mode 0
OUT 83H       ; write it to the control register
IN 80H        ; read the switches from port A
OUT 81H       ; copy them to the LEDs on port B
HLT
```

**The 8255 is a programmable 24-line I/O chip. Write the control word to the control register first (90H here), then use the ports with IN and OUT.** All ports as outputs in mode 0 is 80H, a common exam variant.

## Stop 7: Delay loops and the modulo-N counter

A delay is just a loop that burns a known number of T-states. The standard 16-bit loop:

```
DELAY: LXI D, COUNT    ; 10 T
LOOP:  DCX D           ;  6 T
       MOV A, E        ;  4 T
       ORA D           ;  4 T   (Z = 1 only when D and E are both 0)
       JNZ LOOP        ; 10 T when it jumps, 7 T on the last pass
       RET
```

DCX doesn't set any flags, which is why MOV A, E then ORA D is needed to test for zero.

$$\text{T per loop} = 6 + 4 + 4 + 10 = 24$$

$$\text{Delay} = [10 + 24N - 3] \times T \approx 24N \times T$$

$$N = \frac{\text{Delay}}{24 \times T}$$

> **PYQ, Sessional-2 2023 Q1, 5 marks.** Write an 8085 program to generate a modulo-10 counter with a delay of 90 ms between counts, on a system with a 0.5 µs clock period. Show all calculations.

**Answer:**

$$N = \frac{90\ \text{ms}}{24 \times 0.5\ \mu\text{s}} = \frac{90000}{12} = 7500 = 1D4C_H$$

$$\text{Check: } (7 + 24 \times 7500) \times 0.5\ \mu\text{s} = 90.0035\ \text{ms}$$

```
START: MVI B, 00H      ; count starts at 0
DSPLY: MOV A, B
       OUT PORT1       ; display the count
       CALL DELAY      ; wait 90 ms
       INR B           ; next count
       MOV A, B
       CPI 0AH         ; reached 10?
       JNZ DSPLY       ; no: show it
       JMP START       ; yes: wrap to 0

DELAY: LXI D, 1D4CH    ; N = 7500
LOOP:  DCX D
       MOV A, E
       ORA D
       JNZ LOOP
       RET
```

The count lives in B, not A, because the delay loop overwrites A.

**N = 7500 = 1D4CH. The counter shows 0 to 9, then wraps.** The CALL, RET and counting instructions add a few microseconds, which you may mention and ignore.

**The same program for the other papers' numbers**, changing only COUNT (and CPI for the modulus):

| Paper | Modulus | Delay | N | COUNT | CPI |
|---|---|---|---|---|---|
| Endsem 2025 Q3(c) | 10 | 50 ms | 4167 | 1047H | 0AH |
| Endsem 2024 Q3(c) | 9 | 60 ms | 5000 | 1388H | 09H |
| Sessional-1 2025 Q4 | 8 | 100 ms | 8333 | 208DH | 08H |
| Sessional-2 2023 Q1 | 10 | 90 ms | 7500 | 1D4CH | 0AH |

All four assume a 0.5 µs clock.

That's the run. Four topics, asked the same way every year. Learn them once, and the cat can have your attention back.